

K Nearest Neighbors

This notebook is just to serve as a code reference, please view the lecture video for the full background of the K Nearest Neighbors algorithm. This code is adapted from the ISLR book.

Get Data

We'll use the ISLR package to get the data, you can download it with the code below. Remember to call the library as well.

In [2]:

```
#install.packages('ISLR',repos = 'http://cran.us.r-project.org')
```

In [3]:

```
library(ISLR)
```

We will apply the KNN approach to the Caravan data set, which is part of the ISLR library. This data set includes 85 predictors that measure demographic characteristics for 5,822 individuals. The response variable is Purchase, which indicates whether or not a given individual purchases a Caravan insurance policy. In this data set, only 6% of people purchased caravan insurance.

Let's look at the structure:

In [4]:

```
str(Caravan)
```

```
'data.frame': 5822 obs. of 86 variables:
 $ MOSTYPE : num 33 37 37 9 40 23 39 33 33 11 ...
 $ MAANTHUI: num 1 1 1 1 1 1 2 1 1 2 ...
 $ MGEMOMV : num 3 2 2 3 4 2 3 2 2 3 ...
 $ MGEMLEEF: num 2 2 2 3 2 1 2 3 4 3 ...
 $ MOSHOOFD: num 8 8 8 3 10 5 9 8 8 3 ...
 $ MGODRK : num 0 1 0 2 1 0 2 0 0 3 ...
 $ MGODPR : num 5 4 4 3 4 5 2 7 1 5 ...
 $ MGODOV : num 1 1 2 2 1 0 0 0 3 0 ...
 $ MGODGE : num 3 4 4 4 4 5 5 2 6 2 ...
 $ MRELGE : num 7 6 3 5 7 0 7 7 6 7 ...
 $ MRELSA : num 0 2 2 2 1 6 2 2 0 0 ...
 $ MRELOV : num 2 2 4 2 2 3 0 0 3 2 ...
 $ MFALLEEN: num 1 0 4 2 2 3 0 0 3 2 ...
 $ MFGEKIND: num 2 4 4 3 4 5 3 5 3 2 ...
 $ MFWEKIND: num 6 5 2 4 4 2 6 4 3 6 ...
 $ MOPLHOOG: num 1 0 0 3 5 0 0 0 0 0 ...
 $ MOPLMIDD: num 2 5 5 4 4 5 4 3 1 4 ...
 $ MOPLLAAG: num 7 4 4 2 0 4 5 6 8 5 ...
 $ MBERHOOG: num 1 0 0 4 0 2 0 2 1 2 ...
 $ MBERZELF: num 0 0 0 0 5 0 0 0 1 0 ...
 $ MBERBOER: num 1 0 0 0 4 0 0 0 0 0 ...
 $ MBERMIDD: num 2 5 7 3 0 4 4 2 1 3 ...
 $ MBERARBG: num 5 0 0 1 0 2 1 5 8 3 ...
 $ MBERARBO: num 2 4 2 2 0 2 5 2 1 3 ...
 $ MSKA : num 1 0 0 3 9 2 0 2 1 1 ...
 $ MSKB1 : num 1 2 5 2 0 2 1 1 1 2 ...
 $ MSKB2 : num 2 3 0 1 0 2 4 2 0 1 ...
 $ MSKC : num 6 5 4 4 0 4 5 5 8 4 ...
 $ MSKD : num 1 0 0 0 0 2 0 2 1 2 ...
 $ MHHUUR : num 1 2 7 5 4 9 6 0 9 0 ...
 $ MHKOOP : num 8 7 2 4 5 0 3 9 0 9 ...
 $ MAUT1 : num 8 7 7 9 6 5 8 4 5 6 ...
 $ MAUT2 : num 0 1 0 0 2 3 0 4 2 1 ...
 $ MAUT0 : num 1 2 2 0 1 3 1 2 3 2 ...
 $ MZFONDS : num 8 6 9 7 5 9 9 6 7 6 ...
 $ MZPART : num 1 3 0 2 4 0 0 3 2 3 ...
 $ MINKM30 : num 0 2 4 1 0 5 4 2 7 2 ...
```

```

$ MINK3045: num 4 0 5 5 0 2 3 5 2 3 ...
$ MINK4575: num 5 5 0 3 9 3 3 3 1 3 ...
$ MINK7512: num 0 2 0 0 0 0 0 0 0 1 ...
$ MINK123M: num 0 0 0 0 0 0 0 0 0 0 ...
$ MINKGEM : num 4 5 3 4 6 3 3 3 2 4 ...
$ MKOOPKLA: num 3 4 4 4 3 3 5 3 3 7 ...
$ PWAPART : num 0 2 2 0 0 0 0 0 0 2 ...
$ PWABEDR : num 0 0 0 0 0 0 0 0 0 0 ...
$ PWALAND : num 0 0 0 0 0 0 0 0 0 0 ...
$ PPERSAUT: num 6 0 6 6 0 6 6 0 5 0 ...
$ PBESAUT : num 0 0 0 0 0 0 0 0 0 0 ...
$ PMOTSCO : num 0 0 0 0 0 0 0 0 0 0 ...
$ PVRAAUT : num 0 0 0 0 0 0 0 0 0 0 ...
$ PAANHANG: num 0 0 0 0 0 0 0 0 0 0 ...
$ PTRACTOR: num 0 0 0 0 0 0 0 0 0 0 ...
$ PWERKT : num 0 0 0 0 0 0 0 0 0 0 ...
$ PBROM : num 0 0 0 0 0 0 0 0 3 0 0 ...
$ PLEVEN : num 0 0 0 0 0 0 0 0 0 0 ...
$ PPERSONG: num 0 0 0 0 0 0 0 0 0 0 ...
$ PGEZONG : num 0 0 0 0 0 0 0 0 0 0 ...
$ PWAOREG : num 0 0 0 0 0 0 0 0 0 0 ...
$ PBRAND : num 5 2 2 2 6 0 0 0 0 3 ...
$ PZEILPL : num 0 0 0 0 0 0 0 0 0 0 ...
$ PPLEZIER: num 0 0 0 0 0 0 0 0 0 0 ...
$ PFIETS : num 0 0 0 0 0 0 0 0 0 0 ...
$ PINBOED : num 0 0 0 0 0 0 0 0 0 0 ...
$ PBYSTAND: num 0 0 0 0 0 0 0 0 0 0 ...
$ AWAPART : num 0 2 1 0 0 0 0 0 0 1 ...
$ AWABEDR : num 0 0 0 0 0 0 0 0 0 0 ...
$ AWALAND : num 0 0 0 0 0 0 0 0 0 0 ...
$ APERSAUT: num 1 0 1 1 0 1 1 0 1 0 ...
$ ABESAUT : num 0 0 0 0 0 0 0 0 0 0 ...
$ AMOTSCO : num 0 0 0 0 0 0 0 0 0 0 ...
$ AVRAAUT : num 0 0 0 0 0 0 0 0 0 0 ...
$ AAANHANG: num 0 0 0 0 0 0 0 0 0 0 ...
$ ATRACTOR: num 0 0 0 0 0 0 0 0 0 0 ...
$ AWERKT : num 0 0 0 0 0 0 0 0 0 0 ...
$ ABROM : num 0 0 0 0 0 0 0 0 1 0 0 ...
$ ALEVEN : num 0 0 0 0 0 0 0 0 0 0 ...
$ APERSONG: num 0 0 0 0 0 0 0 0 0 0 ...
$ AGEZONG : num 0 0 0 0 0 0 0 0 0 0 ...
$ AWAOREG : num 0 0 0 0 0 0 0 0 0 0 ...
$ ABRAND : num 1 1 1 1 1 0 0 0 0 1 ...
$ AZEILPL : num 0 0 0 0 0 0 0 0 0 0 ...
$ APLEZIER: num 0 0 0 0 0 0 0 0 0 0 ...
$ AFIETS : num 0 0 0 0 0 0 0 0 0 0 ...
$ AINBOED : num 0 0 0 0 0 0 0 0 0 0 ...
$ ABYSTAND: num 0 0 0 0 0 0 0 0 0 0 ...
$ Purchase: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...

```

In [5]:

```
summary(Caravan$Purchase)
```

Out[5]:

```

No
5474
Yes
348

```

Cleaning Data

Since we are just using this data as a simple example, we won't worry about feature engineering. Let's just remove any NA values by dropping the rows with them.

In [6]:

```
any(is.na(Caravan))
```

```
Out[6]:
```

```
FALSE
```

Looks like we're good! Let's move on to standardize the variable features:

Standardize Variables

Because the KNN classifier predicts the class of a given test observation by identifying the observations that are nearest to it, the scale of the variables matters. Any variables that are on a large scale will have a much larger effect on the distance between the observations, and hence on the KNN classifier, than variables that are on a small scale.

For example, let's check out the variance of two features:

```
In [7]:
```

```
var(Caravan[,1])
```

```
Out[7]:
```

```
165.037847395189
```

```
In [8]:
```

```
var(Caravan[,2])
```

```
Out[8]:
```

```
0.164707781931954
```

Clearly the scales are different! We are now going to standardize all the X variables except Y (Purchase). The Purchase variable is in column 86 of our dataset, so let's save it in a separate variable because the `knn()` function needs it as a separate argument.

```
In [9]:
```

```
# save the Purchase column in a separate variable
purchase <- Caravan[,86]

# Standardize the dataset using "scale()" R function
standardized.Caravan <- scale(Caravan[,-86])
```

Let's check the variance again:

```
In [10]:
```

```
var(standardized.Caravan[,1])
```

```
Out[10]:
```

```
1
```

```
In [11]:
```

```
var(standardized.Caravan[,2])
```

```
Out[11]:
```

```
1
```

We can see that now that all independent variables (X's) have a mean of 1 and standard deviation of 0. Great, then let's divide our dataset into testing and training data. We'll just do a simple split of the first 1000 rows as a test set:

In [12]:

```
# First 100 rows for test set
test.index <- 1:1000
test.data <- standardized.Caravan[test.index,]
test.purchase <- purchase[test.index]
```

In [13]:

```
# Rest of data for training
train.data <- standardized.Caravan[-test.index,]
train.purchase <- purchase[-test.index]
```

Using KNN

Remember that we are trying to come up with a model to predict whether someone will purchase or not. We will use the `knn()` function to do so, and we will focus on 4 of its arguments that we need to specify. The first argument is a data frame that contains the training data set (remember that we don't have the Y here), the second argument is a data frame that contains the testing data set (again no Y variable), the third argument is the `train.purchase` column (Y) that we save earlier, and the fourth argument is the `k` (how many neighbors). Let's start with `k = 1`. `knn()` function returns a vector of predicted Y's.

In [15]:

```
library(class)
```

In [16]:

```
set.seed(101)
predicted.purchase <- knn(train.data, test.data, train.purchase, k=1)
head(predicted.purchase)
```

Out[16]:

```
No No No No No No
```

Now let's evaluate the model we trained and see our misclassification error rate.

In [17]:

```
mean(test.purchase != predicted.purchase)
```

Out[17]:

```
0.116
```

Choosing K Value

Let's see what happens when we choose a different K value:

In [18]:

```
predicted.purchase <- knn(train.data, test.data, train.purchase, k=3)
mean(test.purchase != predicted.purchase)
```

Out[18]:

```
0.073
```

Interesting! Our Misclassification rate went down! What about k=5?

In [19]:

```
predicted.purchase <- knn(train.data,test.data,train.purchase,k=5)
mean(test.purchase != predicted.purchase)
```

Out[19]:

0.066

Should we manually change k and see which k gives us the minimal misclassification rate? NO! we have computers, so let's automate the process with a for() loop. A loop in R repeats the same command as much as you specify. For example, if we want to check for k=1 up to 100, then we have to write 3 x 100 lines of code, but with a for loop, you just need 4 lines of code, and you can repeat those 3 lines up to as many as you want. (Note this may take awhile because you're running the model 20 times!)

[Note on NULL versus NA](#)

In [20]:

```
predicted.purchase = NULL
error.rate = NULL

for(i in 1:20){
  set.seed(101)
  predicted.purchase = knn(train.data,test.data,train.purchase,k=i)
  error.rate[i] = mean(test.purchase != predicted.purchase)
}
```

In [21]:

```
print(error.rate)
```

```
[1] 0.116 0.107 0.074 0.070 0.066 0.064 0.062 0.061 0.058 0.058 0.059 0.058
[13] 0.059 0.059 0.059 0.059 0.059 0.059 0.059 0.059
```

Elbow Method

We can plot out the various error rates for the K values. We should see an "elbow" indicating that we don't get a decrease in error rate for using a higher K. This is a good cut-off point:

In [22]:

```
library(ggplot2)
```

In [23]:

```
k.values <- 1:20
```

In [24]:

```
error.df <- data.frame(error.rate,k.values)
```

In [25]:

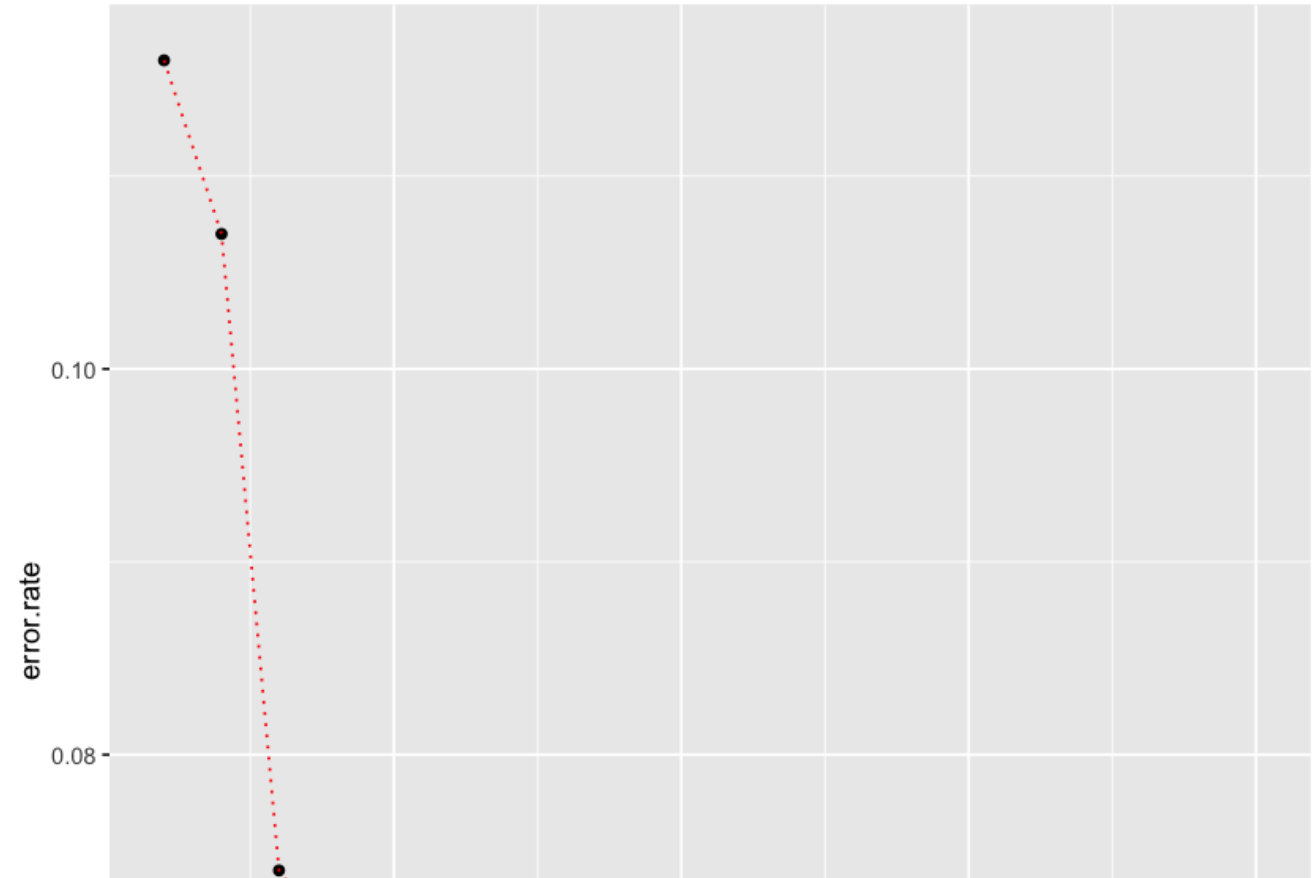
```
error.df
```

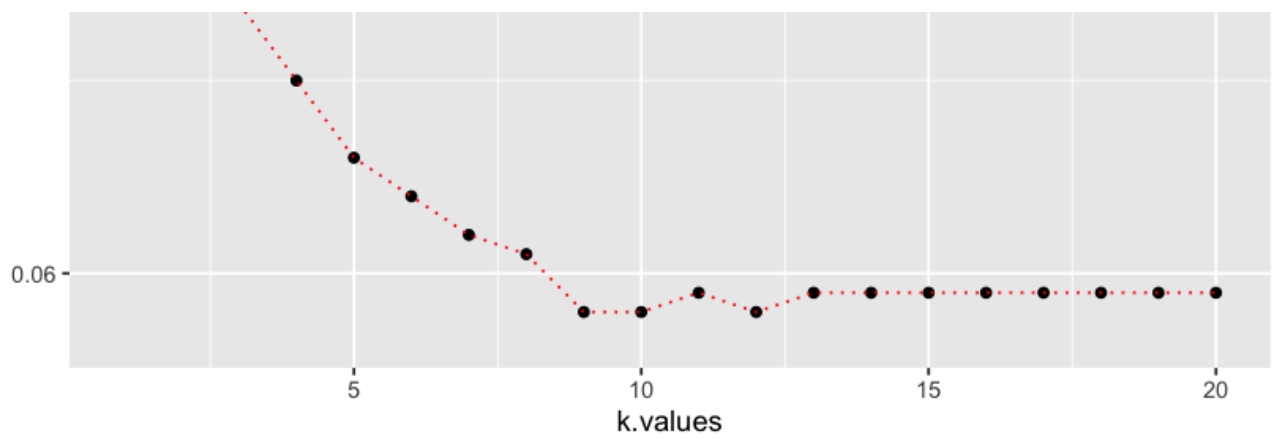
Out[25]:

	error.rate	k.values
1	0.116	1
2	0.107	2
3	0.074	3
4	0.07	4
5	0.066	5
6	0.064	6
7	0.062	7
8	0.061	8
9	0.058	9
10	0.058	10
11	0.059	11
12	0.058	12
13	0.059	13
14	0.059	14
15	0.059	15
16	0.059	16
17	0.059	17
18	0.059	18
19	0.059	19
20	0.059	20

In [26]:

```
ggplot(error.df, aes(x=k.values, y=error.rate)) + geom_point()+ geom_line(lty="dotted", color='red')
```





Here we can clearly see that increasing beyond $K=9$ does not help our misclassification at all. So we can set that as the K for our model during training.

Great Job!